

Example 1: Simple Question Answering

A minimal RAG system demonstrating the core concepts.

What It Does

- Loads a small knowledge base (facts)
- Embeds and stores them in ChromaDB
- Answers questions by retrieving relevant facts
- Optionally uses GPT-3.5 to generate natural answers

Setup

```
# From repository root
cd examples/01-simple-qa

# Install dependencies (if not already installed)
pip install chromadb sentence-transformers openai
```

Usage

With OpenAI (Better Answers)

```
# Set your API key
export OPENAI_API_KEY="your-key-here"
```

```
# Run
python simple_qa.py
```

Without OpenAI (Free, Simple Answers)

```
# Just run it - will use basic fallback
python simple_qa.py
```

Example Output

```
=====
Simple Question Answering System
=====
Loading embedding model...
Adding 6 facts to knowledge base...
✓ Knowledge base updated

=====
Asking Questions
=====

Q: Who created Python?
A: Python was created by Guido van Rossum in 1991.

Sources:
[1] (similarity: 0.823) Python is a high-level programming language created by Guido...
[2] (similarity: 0.234) RAG stands for Retrieval-Augmented Generation, a technique...
-----

Q: What is RAG?
A: RAG stands for Retrieval-Augmented Generation, which is a technique that combines informat

Sources:
[1] (similarity: 0.891) RAG stands for Retrieval-Augmented Generation, a technique...
[2] (similarity: 0.445) Vector embeddings are numerical representations of text...
-----
```

Key Concepts Demonstrated

1. **Embedding:** Text is converted to vectors using Sentence Transformers

2. **Storage:** Vectors are stored in ChromaDB
3. **Retrieval:** Similar vectors are found using cosine similarity
4. **Generation:** Answer is generated from retrieved facts

Code Structure

```
class SimpleQA:
    def add_knowledge(facts)      # Add facts to vector DB
    def answer(question)         # Complete RAG pipeline
    def _generate_with_llm(...)  # Use GPT for generation
    def _generate_simple(...)    # Basic fallback
```

Experiment

Try modifying the code:

1. **Add your own facts:** Change the `knowledge` list
2. **Adjust retrieval:** Change `n_results` to get more/fewer facts
3. **Modify prompt:** Edit the prompt in `_generate_with_llm()`
4. **Try different models:** Use `all-mpnet-base-v2` instead

Next Steps

- See [Example 2](#) for document processing
- See [Example 3](#) for advanced techniques